

第3节. EmailAgent案例

接下来，我们利用前面所学的Runtime和Middleware的知识完成一个实战案例。

1. 需求分析

首先，我们需要弄清楚Email Friend这个agent的功能和实现思路。

1.1 功能特性

Email Friend包含以下核心功能：

- 邮箱认证授权
- 读写邮件
- 邮件操作的人工确认
- 多会话管理
- 基于sse的异步通信

当然，这里的邮箱认证授权、读写操作都是基于Tool模拟的，并非真实操作。目的是通过案例让大家掌握与LangChain的Agent开发相关技能。

通过这个案例希望大家掌握以下技能：

- LangChain中Runtime的使用
- LangChain的中间件使用
 - 利用中间件实现动态提示词
 - 利用中间件实现动态工具选择
- LangChain中agent的异步调用

1.2 预期效果

用户首次访问时需要做认证授权，否则无法进行邮箱操作：



看看我有没有未读邮件

U



好的，我来帮您查看收件箱！查到了！您目前有 **1 封未读邮件**，还有 **1 封已读邮件**。详情如下：

未读邮件

发件人: jane@itcast.cn

主题: 周末见个面？

内容:

嗨 虎哥，
我下周会去城里，不知道我们有没有机会一起喝杯咖啡？
祝好，简

已读邮件

发件人: lixiaolong@itcast.cn

主题: 周五会议

内容:

嗨 虎哥，
非常抱歉，我周五的会议无法准时参加了，能不能重新安排个时间？
祝好，小李

需要我帮您**回复这封未读邮件**吗？或者有其他需要处理的？😊

帮忙回复邮件：

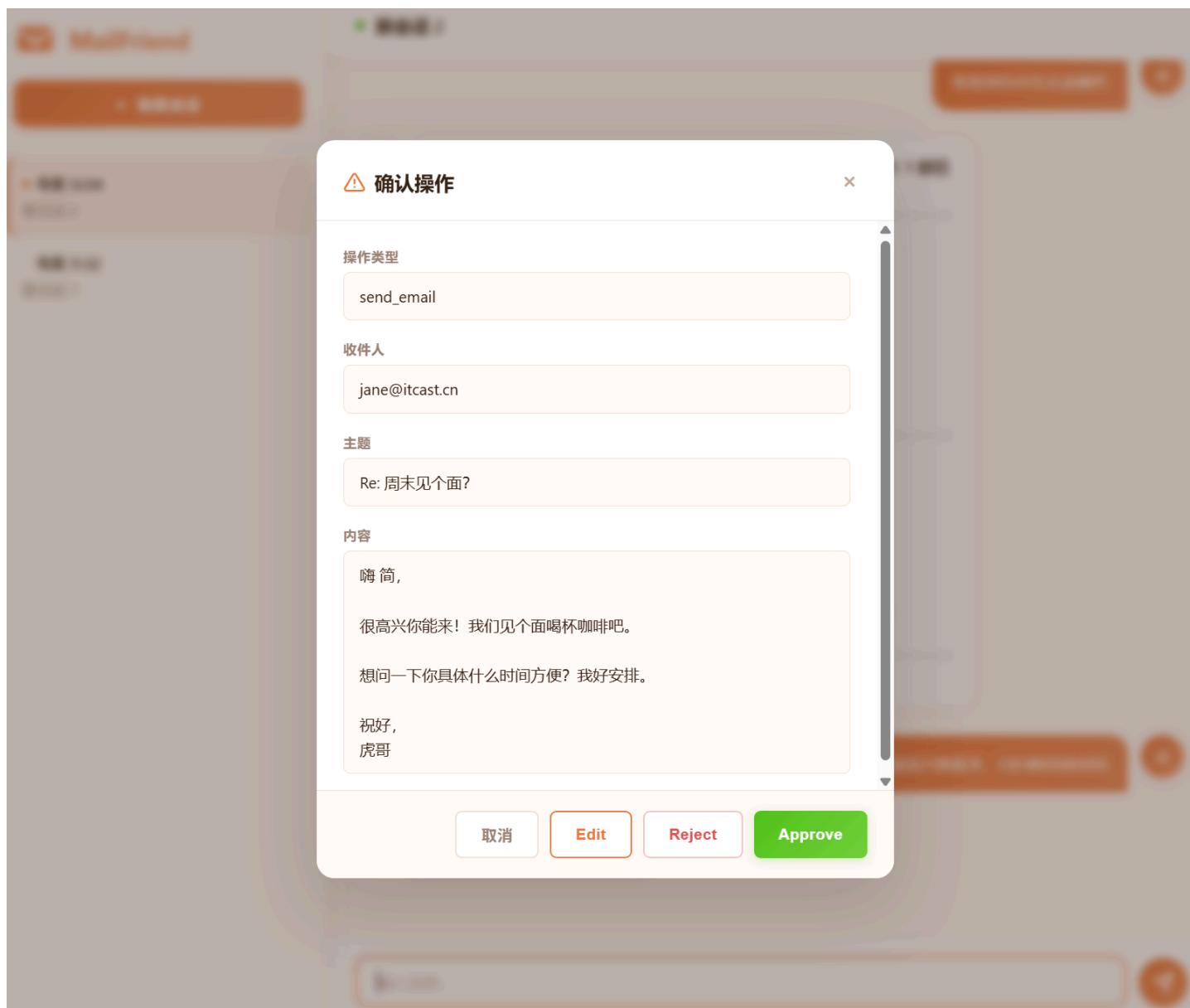
告诉Jane我很高兴她能来，问问她具体时间

U



好的，我来给Jane回复邮件！

但需要经过人工确认：



确认之后，邮件发送成功：



1.3 实现思路分析

之前说过，定义Agent的关键有三部分：

- 模型
- 工具
- 记忆

这个案例不包含多模态内容，我们可以直接使用文本模型。

工具需要定义3个：

- 邮箱鉴权工具
- 查看邮件工具
- 发送邮件工具

记忆，这里我们只需要短期记忆，也就是state，但不能仅仅是message信息，还应该包含用户的鉴权状态。所以我们应该自定义state，包含两个信息：

- messages：会话历史
- authenticated：鉴权状态

另外，为了保证Agent在完成用户鉴权之前不做邮箱等其它操作，我们需要两个不同的系统提示词

- 鉴权提示词：authenticated为False时使用，告知AI核心工作是完成鉴权，否则不能做其它事情
- 邮箱助手提示词：authenticated为True时使用，告知AI核心工作是操作邮箱

然后，我们需要通过中间件实现动态提示词。

与之类似，虽然工具有3个，但为了减少发送给AI的参数信息，我们也分为两种情况：

- 当authenticated为False时：仅提供邮箱鉴权工具
- 当authenticated为True时：仅提供查看邮件、发送邮件工具

2. 功能模拟

接下来，我们依然先使用Jupyter来模拟调试功能。

我们先导入各种所需的依赖：

Code block

```
1 from langchain.agents import create_agent
2 from langchain.messages import AIMessage, HumanMessage, ToolMessage
3 from langgraph.checkpoint.memory import InMemorySaver
4 from langchain.tools import tool, ToolRuntime
5 from langchain.agents import AgentState
6 # 加载环境变量
7 from dotenv import load_dotenv
8
9 load_dotenv()
```

2.1 state

首先，我们要定义一个state，记录用户邮箱授权状态：

Code block

```
1 # state信息，记录用户是否授权邮箱操作
2 class AuthenticatedState(AgentState):
3     authenticated: bool
```

2.2 tool

接着，我们定义3个工具：

- authenticate : 用来模拟用户邮箱鉴权
- check_inbox : 模拟收取邮件
- send_email : 模拟发送邮件

邮箱鉴权工具：

Code block

```
1 # 定义工具，用于用户邮箱鉴权
2 @tool
3 def authenticate(email: str, password: str, runtime: ToolRuntime) -> Command:
4     """Authenticate the user with the given email and password"""
```

```

5
6     # 定义变量，记录校验结果
7     authenticated = False
8     message = "Authentication failed"
9
10    # 校验邮箱和密码
11    if email == "huge@itcast.cn" and password == "123":
12        authenticated = True
13        message = "Successfully authenticated"
14
15    # 返回校验结果
16    return Command(
17        update={
18            "authenticated": authenticated,
19            "messages": [
20                ToolMessage(message, tool_call_id=runtime.tool_call_id)
21            ],
22        }
23    )

```

邮箱操作工具：

Code block

```

1     # 定义工具，收取和发送邮件
2     @tool
3     def check_inbox() -> str:
4         """Read an email from the given address."""
5         # 模拟收件箱邮件
6         return [
7             {
8                 "subject": "周末见个面？",
9                 "content": """
10                    嗨 虎哥，
11                    我下周会去城里，不知道我们有没有机会一起喝杯咖啡？
12
13                    祝好，简
14                """,
15                 "from": "jane@itcast.cn",
16                 "status": "unread"
17             },
18             {
19                 "subject": "周五会议",
20                 "content": """
21                    嗨 虎哥，
22                    非常抱歉，我周五的会议无法准时参加了，能不能重新安排个时间？

```

```

23
24         祝好, 小李
25         """ ,
26         "from": "lixiaolong@itcast.cn",
27         "status": "checked"
28     }
29 ]
30
31
32 @tool
33 def send_email(to: str, subject: str, body: str) -> str:
34     """Send an response email"""
35     return f"邮件已发送至 {to} , 主题: {subject} , 内容: {body}"

```

2.3 动态工具中间件

为了减少调用模型时传递的参数，我们通过自定义中间件实现动态工具：

- 在用户邮箱授权之前，只能看到authenticate工具
- 在用户邮箱授权之后，只能看到send_email和check_inbox工具

Code block

```

1  @wrap_model_call
2  def dynamic_tool_call(
3      request: ModelRequest, handler: Callable[[ModelRequest], ModelResponse]
4  ) -> ModelResponse:
5      """Allow read inbox and send email tools only if user provides correct
6          email and password"""
7
8      # 读取授权状态
9      authenticated = request.state.get("authenticated")
10
11     if authenticated:
12         tools = [check_inbox, send_email]
13     else:
14         tools = [authenticate]
15
16     # 重写工具列表
17     request = request.override(tools=tools)
18     return handler(request)

```


2.4 动态提示词中间件

由于业务分为两个阶段，所以每个阶段的提示词也不一样：

- 鉴权阶段：告知AI的核心工作是鉴权
- 工作阶段：告知AI的核心工作是处理邮件

Code block

```
1  from langchain.agents.middleware import dynamic_prompt
2
3  # 授权前，要求鉴定用户权限的系统提示词
4  unauthenticated_prompt = """You are a helpful email assistant.
5      For system security protocols, you must authenticate user before any other
6      interaction.
7      """
8  # 授权后，邮件处理的系统提示词
9  authenticated_prompt = "You are a helpful assistant that can check the inbox
10     and send emails."
11
12  # 在中间件中通过判断state的authenticated值来动态切换提示词
13  @dynamic_prompt
14  def dynamic_prompt_func(request: ModelRequest) -> str:
15      """Generate system prompt based on authentication status"""
16      authenticated = request.state.get("authenticated")
17      final_prompt = authenticated_prompt if authenticated else
18      unauthenticated_prompt
19      return final_prompt
```

2.5 Agent

最后，就是组织模型、工具、中间件，创建Agent：

Code block

```
1  # 初始化checkpointer
2  checkpointer = InMemorySaver()
3
4  agent = create_agent(
5      "deepseek-chat",
6      tools=[authenticate, check_inbox, send_email],
```

```

7     state_schema=AuthenticatedState,
8     checkpointer=checkpointer,
9     middleware=[
10         dynamic_tool_call,
11         dynamic_prompt_func,
12         HumanInTheLoopMiddleware(
13             interrupt_on={
14                 "authenticate": False,
15                 "check_inbox": False,
16                 "send_email": True,
17             }
18         )
19     ],
20 )

```

2.6 HITL的stream模式

接下来我们测试下Agent效果，这里我们选择采用stream流式调用，在之前我们都采用 `stream_mode` 为 `messages`。

但需要注意的是，由于HITL并不是模型的能力，而是LangChain利用wrap_tool_call在模型调用工具前后的一种拦截行为。因此采用 `messages` 模式只能看到模型返回的消息，无法看到interrupt中断信息。

在HITL存在时，stream采用的模式比较特殊，需要同时包含两种模式：

- `messages`：以token方式返回AI生成的内容
- `updates`：返回Agent调用时的每个步骤的完整信息，比如当出现interrupt时，完整展示interrupt信息

同时，当我们使用的**LangGraph版本大于1.1时**，我们还可以设定stream的 `version` 为 `v2`，这样返回的结果格式会以dict返回，而不是tuple，会更好处理。格式如下：

Code block

```

1  {
2      "type": "interrupt",
3      "data": Interrupt(...),
4      ...
5  }

```

代码如下：

Code block

```
1  config = {"configurable" : {"thread_id": "2"}}
2
3  response = agent.stream(
4      {"messages": [HumanMessage("帮我查看下邮件。")]},
5      config = config,
6      stream_mode= ["messages", "updates"],
7      version="v2"
8  )
9
10 for chunk in response:
11     print(f"-----{chunk["type"]}-----")
12     print(chunk["data"])
```

结果如下：

Code block

```
1  -----messages-----
2  (AIMessageChunk(content=' ', additional_kwargs={}, response_metadata=
3  {'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
4  388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
5  {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
6  3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
7  'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
8  'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
9  513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
10 'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
11 -----messages-----
12 (AIMessageChunk(content='好的', additional_kwargs={}, response_metadata=
13 {'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
14 388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
15 {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
16 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
17 'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
18 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
19 513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
20 'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
21 -----messages-----
22 (AIMessageChunk(content='!', additional_kwargs={}, response_metadata=
23 {'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
24 388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
25 {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
26 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
```

```

'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
7 -----messages-----
8 (AIMessageChunk(content='为了', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
9 -----messages-----
10 (AIMessageChunk(content='查看', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
11 -----messages-----
12 (AIMessageChunk(content='您的', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
13 -----messages-----
14 (AIMessageChunk(content='邮件', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
15 -----messages-----
16 (AIMessageChunk(content='', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-

```

```
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
17 -----messages-----
18 (AIMessageChunk(content='我需要', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
19 -----messages-----
20 (AIMessageChunk(content='先', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
21 -----messages-----
22 (AIMessageChunk(content='验证', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
23 -----messages-----
24 (AIMessageChunk(content='您的', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
```

```
25 -----messages-----
26 (AIMessageChunk(content='身份', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
27 -----messages-----
28 (AIMessageChunk(content='。', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
29 -----messages-----
30 (AIMessageChunk(content='请问', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
31 -----messages-----
32 (AIMessageChunk(content='您', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
33 -----messages-----
34 (AIMessageChunk(content='能', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
```

```

'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
35 -----messages-----
36 (AIMessageChunk(content='提供', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
37 -----messages-----
38 (AIMessageChunk(content='您的', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
39 -----messages-----
40 (AIMessageChunk(content='邮箱', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
41 -----messages-----
42 (AIMessageChunk(content='地址', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
43 -----messages-----
44 (AIMessageChunk(content='和', additional_kwargs={}, response_metadata=
{'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]),

```



```
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step': 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',), 'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name': 'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
```

```
45 -----messages-----
```

```
46 (AIMessageChunk(content='密码', additional_kwargs={}, response_metadata={'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]), {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step': 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',), 'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name': 'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
```

```
47 -----messages-----
```

```
48 (AIMessageChunk(content='吗', additional_kwargs={}, response_metadata={'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]), {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step': 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',), 'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name': 'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
```

```
49 -----messages-----
```

```
50 (AIMessageChunk(content='? ', additional_kwargs={}, response_metadata={'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-388fc5bd1733', tool_calls=[], invalid_tool_calls=[], tool_call_chunks=[]), {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step': 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',), 'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name': 'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
```

```
51 -----messages-----
```

```
52 (AIMessageChunk(content='', additional_kwargs={}, response_metadata={'finish_reason': 'stop', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'}, id='lc_run--019e0c10-f203-7891-956a-388fc5bd1733', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 321, 'output_tokens': 25, 'total_tokens': 346, 'input_token_details': {'cache_read': 256}, 'output_token_details': {}}, tool_call_chunks=[]), {'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step': 3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',), 'langgraph_path': ('__pregel_pull', 'model'),
```



```

'langgraph_checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3',
'checkpoint_ns': 'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'ls_provider':
'deepseek', 'ls_model_name': 'deepseek-chat', 'ls_model_type': 'chat',
'ls_temperature': None})
53 -----messages-----
54 (AIMessageChunk(content='', additional_kwargs={}, response_metadata={},
id='lc_run--019e0c10-f203-7891-956a-388fc5bd1733', tool_calls=[],
invalid_tool_calls=[], tool_call_chunks=[], chunk_position='last'),
{'ls_integration': 'langchain_chat_model', 'thread_id': '1', 'langgraph_step':
3, 'langgraph_node': 'model', 'langgraph_triggers': ('branch:to:model',),
'langgraph_path': ('__pregel_pull', 'model'), 'langgraph_checkpoint_ns':
'model:bf14c4b8-513a-e109-9a79-05b53a3364b3', 'checkpoint_ns': 'model:bf14c4b8-
513a-e109-9a79-05b53a3364b3', 'ls_provider': 'deepseek', 'ls_model_name':
'deepseek-chat', 'ls_model_type': 'chat', 'ls_temperature': None})
55 -----updates-----
56 {'model': {'messages': [AIMessage(content='好的! 为了查看您的邮件, 我需要先验证您的
身份。请问您能提供您的邮箱地址和密码吗?', additional_kwargs={}, response_metadata=
{'finish_reason': 'stop', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402',
'model_provider': 'deepseek'}], id='lc_run--019e0c10-f203-7891-956a-
388fc5bd1733', tool_calls=[], invalid_tool_calls=[], usage_metadata=
{'input_tokens': 321, 'output_tokens': 25, 'total_tokens': 346,
'input_token_details': {'cache_read': 256}, 'output_token_details': {}})]}}
57 -----updates-----
58 {'HumanInTheLoopMiddleware.after_model': None}

```

可以看到，我要求查看邮件，但AI并不会这么做，而是会询问我的邮箱地址和密码，要求完成授权。

2.7 测试

接下来，我们继续测试hitl过程

2.7.1 邮箱授权

我们继续调用Agent，传递用户邮箱和密码。不过接下来打印的时候我们就可以把message类型和updates类型分开打印了：

Code block

```

1  # 用户授权
2  response = agent.stream(
3      {"messages": [HumanMessage("huge@itcast.cn/123")]},
4      config = config,
5      stream_mode= ["messages", "updates"],

```

```

6     version="v2"
7 )
8
9 def print_chunk(chunk: dict[str, Any]):
10     type = chunk["type"]
11     data = chunk["data"]
12     if type == "messages":
13         token, metadata = data
14         if isinstance(token, AIMessage) and token.content:
15             print(token.content, end="", flush=True)
16     elif type == "updates":
17         print("")
18         print("=====updates=====")
19         print(data)
20
21 for chunk in response:
22     print_chunk(chunk)

```

运行结果太长，我们截取一部分：

Code block

```

1  好的，我来帮您验证身份。
2  =====updates=====
3  {'model': {'messages': [AIMessage(content='好的，我来帮您验证身份。',
4  additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls',
5  'model_name': 'deepseek-v4-flash', 'system_fingerprint':
6  'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
7  id='lc_run--019e0c0a-e121-7f52-b764-f15acb5d46dd', tool_calls=[{'name':
8  'authenticate', 'args': {'email': 'huge@itcast.cn', 'password': '123'}, 'id':
9  'call_00_S48AaYVcrXdbEGN4oor17885', 'type': 'tool_call'}], invalid_tool_calls=
10 [], usage_metadata={'input_tokens': 345, 'output_tokens': 72, 'total_tokens':
11 417, 'input_token_details': {'cache_read': 256}, 'output_token_details':
12 {}})]}]}
13
14  =====updates=====
15  {'HumanInTheLoopMiddleware.after_model': None}
16
17  =====updates=====
18  {'tools': {'authenticated': True, 'messages':
19  [ToolMessage(content='Successfully authenticated', name='authenticate',
20  id='5d04f995-ddfc-44a6-b92e-180cb00b55b3',
21  tool_call_id='call_00_S48AaYVcrXdbEGN4oor17885')]}]}
22  验证成功！现在我来帮您查看邮件。
23  =====updates=====

```

```
12 {'model': {'messages': [AIMessage(content='验证成功! 现在我来帮您查看邮件。',
additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls',
'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c0a-e7b5-7903-b1fc-196f6e3ade16', tool_calls=[{'name':
'check_inbox', 'args': {}, 'id': 'call_00_nAsNhsFbZ8raGbHPQcJ14602', 'type':
'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 462,
'output_tokens': 37, 'total_tokens': 499, 'input_token_details':
{'cache_read': 128, 'output_token_details': {}})]}]}
```

13

14 =====updates=====

15 {'HumanInTheLoopMiddleware.after_model': None}

16

17 =====updates=====

18 {'tools': {'messages': [ToolMessage(content='[{"subject": "周末见个面?",
"content": "\\n 嗨 虎哥,\\n 我下周会去城里, 不知道我
们有没有机会一起喝杯咖啡? \\n\\n 祝好, 简\\n ", "from":
"jane@itcast.cn", "status": "unread"}, {"subject": "周五会议", "content": "\\n
 嗨 虎哥,\\n 非常抱歉, 我周五的会议无法准时参加了, 能不能
重新安排个时间? \\n\\n 祝好, 小李\\n ", "from":
"lixiaolong@itcast.cn", "status": "checked"}]'], name='check_inbox',
id='0b2e75ef-1260-4ae2-8b92-5e29c542a970',
tool_call_id='call_00_nAsNhsFbZ8raGbHPQcJ14602')]]}

19 您有 **2 封邮件**:

20

21 ---

22

23 📩 **第1封 (未读) **

24 - **发件人:** jane@itcast.cn

25 - **主题:** 周末见个面?

26 - **内容:** 简说下周会去城里, 想问问有没有机会一起喝杯咖啡。

27

28 📩 **第2封 (已读) **

29 - **发件人:** lixiaolong@itcast.cn

30 - **主题:** 周五会议

31 - **内容:** 小李说周五的会议无法准时参加, 想重新安排时间。

32

33 ---

34

35 请问您需要回复哪封邮件吗?

36 =====updates=====

37 {'model': {'messages': [AIMessage(content='您有 **2 封邮件**:\n\n---\n\n📩 **第1
封 (未读) **\n- **发件人:** jane@itcast.cn\n- **主题:** 周末见个面? \n- **内容:**
** 简说下周会去城里, 想问问有没有机会一起喝杯咖啡。\\n\n📩 **第2封 (已读) **\n- **发件
人:** lixiaolong@itcast.cn\n- **主题:** 周五会议\n- **内容:** 小李说周五的会议无法
准时参加, 想重新安排时间。\\n\n---\n\n请问您需要回复哪封邮件吗?', additional_kwargs=
{}, response_metadata={'finish_reason': 'stop', 'model_name': 'deepseek-v4-

```

flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402',
'model_provider': 'deepseek'}, id='lc_run--019e0c0a-ed81-7641-9233-
a088a6af7ac5', tool_calls=[], invalid_tool_calls=[], usage_metadata=
{'input_tokens': 648, 'output_tokens': 134, 'total_tokens': 782,
'input_token_details': {'cache_read': 384}, 'output_token_details': {}})]}]
38
39 =====updates=====
40 {'HumanInTheLoopMiddleware.after_model': None}

```

可以看到，Agent在鉴权成功后，帮我查看了邮件内容，并且询问我是否要回复邮件。

2.7.2 触发Interrupt

我们继续调用Agent，让它帮我回复邮件，不过接下来打印的时候我们就可以把message类型和updates类型分开打印了：

Code block

```

1  # 用户授权
2  response = agent.stream(
3      {"messages": [HumanMessage("huge@itcast.cn/123")]},
4      config = config,
5      stream_mode= ["messages", "updates"],
6      version="v2"
7  )
8
9  for chunk in response:
10     print_chunk(chunk)

```

运行结果如下：

Code block

```

1  好的，我来帮您回复简的邮件。
2  =====updates=====
3  {'model': {'messages': [AIMessage(content='好的，我来帮您回复简的邮件。',
additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls',
'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c12-dfbd-7531-98f0-3553a5a606e7', tool_calls=[{'name':
'send_email', 'args': {'to': 'jane@itcast.cn', 'subject': 'Re: 周末见个面？',
'body': '嗨 简，\n\n很高兴你能来！我很期待和你一起喝杯咖啡。\n\n请问你具体什么时间方便
呢？我们可以一起定个合适的时间。\n\n祝好，\n虎哥'}], 'id':

```

```

'call_00_DL1oBpIe7Rqb26eT9A1o6403', 'type': 'tool_call'}], invalid_tool_calls=
[], usage_metadata={'input_tokens': 874, 'output_tokens': 131, 'total_tokens':
1005, 'input_token_details': {'cache_read': 768}, 'output_token_details':
{}})}}}}
4
5 =====updates=====
6 {'__interrupt__': (Interrupt(value={'action_requests': [{'name': 'send_email',
'args': {'to': 'jane@itcast.cn', 'subject': 'Re: 周末见个面?', 'body': '嗨 简,
\\n\\n很高兴你能来! 我很期待和你一起喝杯咖啡.\\n\\n请问你具体什么时间方便呢? 我们可以一起定
个合适的时间.\\n\\n祝好,\\n虎哥'}, 'description': "Tool execution requires
approval\\n\\nTool: send_email\\nArgs: {'to': 'jane@itcast.cn', 'subject': 'Re: 周
末见个面?', 'body': '嗨 简, \\n\\n很高兴你能来! 我很期待和你一起喝杯咖啡.\\n\\n请问你
具体什么时间方便呢? 我们可以一起定个合适的时间.\\n\\n祝好,\\n虎哥'}"]},
'review_configs': [{'action_name': 'send_email', 'allowed_decisions':
['approve', 'edit', 'reject', 'respond']}]}],
id='befc16a002eee54b0e0d44123c64a0f5'),)}

```

可以看到，虽然AI已经帮我们写了邮件，但是并没有发送，而是在调用send_email这个Tool时被拦截了，同时在updates中可以看到interrupt信息。

接下来，就可以发送到前端，让用户来做决定了。

2.7.3 用户介入

接下来，用户就可以根据自己的需求来处理interrupt了，可以选择：

- Approve
- Reject
- Edit

例如，我们选择reject拒绝发送，同时告诉AI邮件内容有问题，让它改一改：

Code block

```

1  # 用户处理interrupt, 比如reject
2  response = agent.stream(
3      Command(
4          resume={
5              "decisions": [
6                  {
7                      "type": "reject",
8                      # 告知AI拒绝的原因
9                      "message": "语气太书面了，这不是工作邮件，Jane是我的好朋友，回复
自然一点。"

```

```

10         }
11     ]
12 }
13 ),
14 config = config,
15 stream_mode= ["messages", "updates"],
16 version="v2"
17 )
18
19 for chunk in response:
20     print_chunk(chunk)

```

运行结果：

Code block

```

1
2  =====updates=====
3  {'HumanInTheLoopMiddleware.after_model': {'messages': [AIMessage(content='好的，
我来帮您回复简的邮件。', additional_kwargs={}, response_metadata=
{'finish_reason': 'tool_calls', 'model_name': 'deepseek-v4-flash',
'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402',
'model_provider': 'deepseek'}, id='lc_run--019e0c12-dfbd-7531-98f0-
3553a5a606e7', tool_calls=[{'name': 'send_email', 'args': {'to':
'jane@itcast.cn', 'subject': 'Re: 周末见个面？', 'body': '嗨 简，\n\n很高兴你能来！
我很期待和你一起喝杯咖啡。\n\n请问你具体什么时间方便呢？我们可以一起定个合适的时间。\n\n
祝好，\n虎哥'}], 'id': 'call_00_DL1oBpIe7Rqb26eT9A1o6403', 'type':
'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 874,
'output_tokens': 131, 'total_tokens': 1005, 'input_token_details':
{'cache_read': 768}, 'output_token_details': {}})], ToolMessage(content='语气太书
面了，这不是工作邮件，Jane是我的好朋友，回复自然一点。', name='send_email',
id='ee72e747-1120-4f7c-ba7c-9628321a7a39',
tool_call_id='call_00_DL1oBpIe7Rqb26eT9A1o6403', status='error')]]}
4 好的，我换一种更轻松自然的语气重新发送。
5  =====updates=====
6  {'model': {'messages': [AIMessage(content='好的，我换一种更轻松自然的语气重新发
送。', additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls',
'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c17-a8a5-7a52-9c60-73511c5b41ab', tool_calls=[{'name':
'send_email', 'args': {'to': 'jane@itcast.cn', 'subject': 'Re: 周末见个面？',
'body': '嗨 Jane，\n\n太好了！好久不见，必须约起来！☕️\n\n你具体啥时候方便？跟我说说时
间，我来安排！\n\n等你消息哈～\n\n虎哥'}], 'id':
'call_00_6mCcTtLNUOG1yUoMDcMf5621', 'type': 'tool_call'}], invalid_tool_calls=
[], usage_metadata={'input_tokens': 1033, 'output_tokens': 131,

```

```

'total_tokens': 1164, 'input_token_details': {'cache_read': 896},
'output_token_details': {}})]}]
7
8 =====updates=====
9 {'__interrupt__': (Interrupt(value={'action_requests': [{'name': 'send_email',
'args': {'to': 'jane@itcast.cn', 'subject': 'Re: 周末见个面? ', 'body': '嗨
Jane, \n\n太好了! 好久不见, 必须约起来! ☕\n\n你具体啥时候方便? 跟我说说时间, 我来安排!
\n\n等你消息哈~\n\n虎哥'}, 'description': "Tool execution requires
approval\n\nTool: send_email\nArgs: {'to': 'jane@itcast.cn', 'subject': 'Re: 周
末见个面? ', 'body': '嗨 Jane, \n\n太好了! 好久不见, 必须约起来! ☕\n\n你具体啥时
候方便? 跟我说说时间, 我来安排! \n\n等你消息哈~\n\n虎哥'}"}]], 'review_configs':
[{'action_name': 'send_email', 'allowed_decisions': ['approve', 'edit',
'reject', 'respond']}]}, id='838dbb7f8c6b33cbab150adb1e2bc645'),)}

```

可以看到, 我们reject的理由会作为ToolMessage返回给AI, AI看到后重写了一封邮件, 再次尝试发送, 再次触发了Interrupt

接下来, 就可以再次让人工介入了。

2.8 会话历史

带有hitl的Agent会话历史也比较特殊, 与之前的纯对话历史不同。

2.8.1 checkpointer获取历史

之前我们通过checkpointer获取会话历史, 但是存在一个问题, checkpointer中直接获取拿不到interrupt信息, 只能拿到Message信息:

Code block

```
1 checkpointer.get(config)
```

结果:

Code block

```

1 {'v': 4,
2  'ts': '2026-05-09T09:41:36.374100+00:00',
3  'id': '1f14b8b4-57fd-6d94-8011-d3cf9dd664b9',

```

```

4      'channel_versions': {'__start__':
    '0000000000000000000000000000000016.0.8655555881670611',
5      'messages': '0000000000000000000000000000000019.0.7544398364008247',
6      'branch:to:model': '0000000000000000000000000000000019.0.7544398364008247',
7      'branch:to:HumanInTheLoopMiddleware.after_model':
    '0000000000000000000000000000000019.0.7544398364008247',
8      '__pregel_tasks': '0000000000000000000000000000000012.0.2455949578797224',
9      'authenticated': '0000000000000000000000000000000009.0.4573035367639653'},
10     'versions_seen': {'__input__': {}},
11     '__start__': {'__start__':
    '0000000000000000000000000000000015.0.15918202817556004'},
12     'model': {'branch:to:model':
    '0000000000000000000000000000000018.0.38132983800910747'},
13     'HumanInTheLoopMiddleware.after_model':
    {'branch:to:HumanInTheLoopMiddleware.after_model':
    '0000000000000000000000000000000017.0.31280029323537684'},
14     'tools': {},
15     '__interrupt__': {'messages':
    '0000000000000000000000000000000017.0.31280029323537684',
16     'authenticated': '0000000000000000000000000000000009.0.4573035367639653',
17     '__start__': '0000000000000000000000000000000016.0.8655555881670611',
18     '__pregel_tasks': '0000000000000000000000000000000012.0.2455949578797224',
19     'branch:to:model': '0000000000000000000000000000000017.0.31280029323537684',
20     'branch:to:HumanInTheLoopMiddleware.after_model':
    '0000000000000000000000000000000017.0.31280029323537684'}}},
21     'updated_channels': ['branch:to:HumanInTheLoopMiddleware.after_model',
22     'messages'],
23     'channel_values': {'messages': [HumanMessage(content='帮我查看下邮件。',
additional_kwargs={}, response_metadata={}, id='da4973b9-154e-4dfa-afb7-
b54927f46e85'),
24         AIMessage(content='我需要先验证您的身份才能帮您查看邮件。请提供您的邮箱地址和密码以
便进行身份验证。', additional_kwargs={}, response_metadata={'finish_reason':
'stop', 'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c1c-71b0-7ca3-a262-f8b6278a48da', tool_calls=[],
invalid_tool_calls=[], usage_metadata={'input_tokens': 316, 'output_tokens':
24, 'total_tokens': 340, 'input_token_details': {'cache_read': 256},
'output_token_details': {}}),
        HumanMessage(content='huge@itcast.cn/123', additional_kwargs={},
response_metadata={}, id='758a2126-9ec9-44de-abdf-f24acc6dd85d'),
        AIMessage(content='好的，让我验证您的身份。', additional_kwargs={},
response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-v4-
flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402',
'model_provider': 'deepseek'}, id='lc_run--019e0c1c-aa00-7711-a2f0-
0f7e09199e4f', tool_calls=[{'name': 'authenticate', 'args': {'email':
'huge@itcast.cn', 'password': '123'}}, {'call_00_cwKozzKxNBY7FTSDn8XU7337', 'type': 'tool_call'}], invalid_tool_calls=

```



```
[], usage_metadata={'input_tokens': 351, 'output_tokens': 71, 'total_tokens': 422, 'input_token_details': {'cache_read': 256}, 'output_token_details': {})),
27     ToolMessage(content='Successfully authenticated', name='authenticate',
id='a37f35bc-b8fb-4264-b4b5-369e5b55a942',
tool_call_id='call_00_cwKozzKxNBY7FTSDn8XU7337'),
28     AIMessage(content='身份验证成功! 现在让我查看您的收件箱。', additional_kwargs=
{}, response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-
v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c1c-b1e8-7ce0-a72e-3a11880cad1d', tool_calls=[{'name':
'check_inbox', 'args': {}, 'id': 'call_00_T0Hqh8wGpGWmGSFQcxae5068', 'type':
'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 467,
'output_tokens': 39, 'total_tokens': 506, 'input_token_details':
{'cache_read': 128}, 'output_token_details': {})),
29     ToolMessage(content='[{"subject": "周末见个面?", "content": "\n\n
嗨 虎哥, \n\n          我下周会去城里, 不知道我们有没有机会一起喝杯咖啡?
\n\n\n          祝好, 简\n\n          ", "from": "jane@itcast.cn",
"status": "unread"}, {"subject": "周五会议", "content": "\n\n          嗨 虎
哥, \n\n          非常抱歉, 我周五的会议无法准时参加了, 能不能重新安排个时间?
\n\n\n          祝好, 小李\n\n          ", "from":
"lixiaolong@itcast.cn", "status": "checked"}]', name='check_inbox',
id='efde42b2-d17c-4782-af35-bece76382a86',
tool_call_id='call_00_T0Hqh8wGpGWmGSFQcxae5068'),
30     AIMessage(content='好的, 您的收件箱中有以下邮件: \n\n---\n\n### 📧 邮件列表
\n\n**1. 来自: ** `jane@itcast.cn` (简) \n\n**主题: ** 周末见个面? \n\n**状态: ** 📧 未
读\n\n**内容: **\n> 嗨 虎哥, \n> 我下周会去城里, 不知道我们有没有机会一起喝杯咖啡?
\n>\n> 祝好, 简\n\n---\n\n**2. 来自: ** `lixiaolong@itcast.cn` (小李) \n\n**主题: **
周五会议\n\n**状态: ** ✅ 已读\n\n**内容: **\n> 嗨 虎哥, \n> 非常抱歉, 我周五的会议无法准时
参加了, 能不能重新安排个时间? \n>\n> 祝好, 小李\n\n---\n\n您有一封来自 **简
(jane@itcast.cn)** 的未读邮件, 她想约您喝咖啡。您需要我回复哪封邮件吗? ',
additional_kwargs={}, response_metadata={'finish_reason': 'stop',
'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c1c-b6e8-7980-a1b7-2a5ee112693b', tool_calls=[],
invalid_tool_calls=[], usage_metadata={'input_tokens': 655, 'output_tokens':
204, 'total_tokens': 859, 'input_token_details': {'cache_read': 384},
'output_token_details': {})),
31     HumanMessage(content='帮我回复Jane, 很高兴她能来, 问问她具体见面时间',
additional_kwargs={}, response_metadata={}, id='34d17f0c-a7bd-43a1-ab48-
6c240ce14f1a'),
32     AIMessage(content='好的, 我来回复Jane的邮件。', additional_kwargs={},
response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-v4-
flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402',
'model_provider': 'deepseek'}, id='lc_run--019e0c1c-ea90-72d2-8d23-
b0a843d6c6c1', tool_calls=[{'name': 'send_email', 'args': {'to':
'jane@itcast.cn', 'subject': 'Re: 周末见个面?', 'body': '嗨 简, \n\n很高兴你要来城
里! 我们当然可以一起喝杯咖啡。 \n\n请问你具体什么时间方便见面呢? \n\n期待见面! \n\n祝好,
```

```

\n虎哥'}, 'id': 'call_00_2WAI8S2tKU5FHdJW7PNL3802', 'type': 'tool_call']],
invalid_tool_calls=[], usage_metadata={'input_tokens': 876, 'output_tokens':
128, 'total_tokens': 1004, 'input_token_details': {'cache_read': 768},
'output_token_details': {}}),
33     ToolMessage(content='语气太书面了，这不是工作邮件，Jane是我的好朋友，回复自然一
点。', name='send_email', id='b32acef4-fa75-45e5-892e-68ae067e16e1',
tool_call_id='call_00_2WAI8S2tKU5FHdJW7PNL3802', status='error'),
34     AIMessage(content='明白了，这是好朋友之间的邮件，我重新给您写一封更自然的。',
additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls',
'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0c1d-176d-7113-a5aa-b3327e431392', tool_calls=[{'name':
'send_email', 'args': {'subject': 'Re: 周末见个面?', 'body': '嗨 简，\n\n太好了！
你要来城里啦，必须约起来喝杯咖啡！\n\n你具体啥时候方便呀？跟我说个时间呗~\n\n回头见！\n虎
哥', 'to': 'jane@itcast.cn'}], 'id': 'call_00_wDU2A4kFNKRvYng4rnxo8221',
'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens':
1031, 'output_tokens': 135, 'total_tokens': 1166, 'input_token_details':
{'cache_read': 768}, 'output_token_details': {}})],
35     'branch:to:HumanInTheLoopMiddleware.after_model': None,
36     '__pregel_tasks': [],
37     'authenticated': True}}

```

2.8.2 state获取历史

如果要同时获取到历史消息、interrupt等信息，必须通过AgentState:

Code block

```
1 agent.get_state(config)
```

结果:

Code block

```

1 StateSnapshot(values={'messages': [HumanMessage(content='帮我查看下邮件。',
additional_kwargs={}, response_metadata={}, id='9d0e51c5-f322-4c9e-88fb-
5a5b43115374'), AIMessage(content='好的，我先帮您进行身份验证。请问您的邮箱地址和密码
是什么?', additional_kwargs={}, response_metadata={'finish_reason': 'stop',
'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
id='lc_run--019e0186-8a2c-74a0-9704-f14ab790d49b', tool_calls=[],
invalid_tool_calls=[], usage_metadata={'input_tokens': 316, 'output_tokens':
18, 'total_tokens': 334, 'input_token_details': {'cache_read': 256},
'output_token_details': {}}), HumanMessage(content='huge@itcast.cn/123',

```

```
additional_kwargs={}, response_metadata={}, id='30ce13f3-fb3b-4595-a459-c1c8a858ac57'), AIMessage(content='好的，我来帮您进行身份验证。', additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'}, id='lc_run--019e0186-b8ea-7cd3-aa3e-5854bc67f515', tool_calls=[{'name': 'authenticate', 'args': {'email': 'huge@itcast.cn', 'password': '123'}, 'id': 'call_00_i3hmccq6yJ7VN3zWV4WY5449', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 345, 'output_tokens': 73, 'total_tokens': 418, 'input_token_details': {'cache_read': 256}, 'output_token_details': {}}), ToolMessage(content='Successfully authenticated', name='authenticate', id='e615a53e-9d57-4441-8833-f339370ef9ab', tool_call_id='call_00_i3hmccq6yJ7VN3zWV4WY5449'), AIMessage(content='太好了，身份验证成功！我来帮您查看邮件。', additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'}, id='lc_run--019e0186-bfec-7050-8a99-f8cfec6e3e0', tool_calls=[{'name': 'check_inbox', 'args': {}, 'id': 'call_00_ol2PU4erxIj6Sko9Gxen6317', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 463, 'output_tokens': 39, 'total_tokens': 502, 'input_token_details': {'cache_read': 128}, 'output_token_details': {}}), ToolMessage(content='[{"subject": "周末见个面?", "content": "\n\n嗨 虎哥,\n\n我下周会去城里，不知道我们有没有机会一起喝杯咖啡?\n\n\n\n祝好，简\n\n", "from": "jane@itcast.cn", "status": "unread"}, {"subject": "周五会议", "content": "\n\n嗨 虎哥,\n\n非常抱歉，我周五的会议无法准时参加了，能不能重新安排个时间?\n\n\n\n祝好，小李\n\n", "from": "lixiaolong@itcast.cn", "status": "checked"}]', name='check_inbox', id='a7a99501-f953-4042-b6ee-12242b1559ff', tool_call_id='call_00_ol2PU4erxIj6Sko9Gxen6317'), AIMessage(content='以下是您的收件箱内容，共有 **2封邮件**：\n\n---\n\n### 📧 第1封邮件（未读）\n发件人：jane@itcast.cn（简）\n主题：**周末见个面？**\n内容：**\n> 嗨 虎哥,\n> 我下周会去城里，不知道我们有没有机会一起喝杯咖啡?\n> \n> 祝好，简\n\n### 📧 第2封邮件（已读）\n发件人：lixiaolong@itcast.cn（小李）\n主题：**周五会议**\n内容：**\n> 嗨 虎哥,\n> 非常抱歉，我周五的会议无法准时参加了，能不能重新安排个时间?\n> \n> 祝好，小李\n\n---\n\n请问您需要回复哪一封邮件?', additional_kwargs={}, response_metadata={'finish_reason': 'stop', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'}, id='lc_run--019e0186-c56e-7803-8e3d-1cf4f10dff3b', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 651, 'output_tokens': 188, 'total_tokens': 839, 'input_token_details': {'cache_read': 384}, 'output_token_details': {}}), HumanMessage(content='帮我回复Jane，很高兴她能来，问问她具体见面时间', additional_kwargs={}, response_metadata={}, id='8b29113d-397d-48e8-b949-ae5272c8e68b'), AIMessage(content='好的，我来给Jane回复邮件。', additional_kwargs={}, response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},
```

```
id='lc_run--019e0186-f998-76b1-b3a4-2b10107ce0f7', tool_calls=[{'name':  
'send_email', 'args': {'to': 'jane@itcast.cn', 'subject': 'Re: 周末见个面?',  
'body': '嗨 简, \n\n很高兴听说你要来城里! 我们当然可以一起喝杯咖啡聊聊. \n\n请问你具体什  
么时间方便呢? 我们可以约个时间见面. \n\n期待见到你! \n\n祝好, \n虎哥'}], 'id':  
'call_00_joBsSVVaI3XCElJPou4L6453', 'type': 'tool_call'}], invalid_tool_calls=  
[], usage_metadata={'input_tokens': 856, 'output_tokens': 136, 'total_tokens':  
992, 'input_token_details': {'cache_read': 768}, 'output_token_details': {})),  
ToolMessage(content='语气太书面了, 这不是工作邮件, Jane是我的好朋友, 回复自然一点.',  
name='send_email', id='92520d05-3494-47b2-8e4a-369038a49bd4',  
tool_call_id='call_00_joBsSVVaI3XCElJPou4L6453', status='error'),  
AIMessage(content='好的, 我来重新发一封更自然、更口语化的回复!', additional_kwargs=  
{}, response_metadata={'finish_reason': 'tool_calls', 'model_name': 'deepseek-  
v4-flash', 'system_fingerprint':  
'fp_058df29938_prod0820_fp8_kvcache_20260402', 'model_provider': 'deepseek'},  
id='lc_run--019e0187-2575-7f80-a5a2-3877bc425e92', tool_calls=[{'name':  
'send_email', 'args': {'subject': 'Re: 周末见个面?', 'to': 'jane@itcast.cn',  
'body': '嘿 Jane! \n\n当然好啊, 好久不见! 你什么时候到? 具体哪天有空, 咱们约个时间喝咖  
啡! \n\n等你消息哈~\n\n虎哥'}], 'id': 'call_00_pz3xlFS5s1Af5vjqTPgh9771', 'type':  
'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 1019,  
'output_tokens': 129, 'total_tokens': 1148, 'input_token_details':  
{'cache_read': 768}, 'output_token_details': {}))), 'authenticated': True},  
next=('HumanInTheLoopMiddleware.after_model',), config={'configurable':  
{'thread_id': '2', 'checkpoint_ns': '', 'checkpoint_id': '1f149edc-4374-63d5-  
8011-44f075bb470a'}}, metadata={'source': 'loop', 'step': 17, 'parents': {},  
'ls_integration': 'langchain_create_agent'}, created_at='2026-05-  
07T08:21:37.322889+00:00', parent_config={'configurable': {'thread_id': '2',  
'checkpoint_ns': '', 'checkpoint_id': '1f149edc-2a81-6141-8010-  
5e7f6ce70e4e'}}, tasks=(PregelTask(id='3d2c8cf9-4d91-f0f0-a06e-db17304d5edb',  
name='HumanInTheLoopMiddleware.after_model', path=('__pregel_pull',  
'HumanInTheLoopMiddleware.after_model'), error=None, interrupts=  
(Interrupt(value={'action_requests': [{'name': 'send_email', 'args':  
{'subject': 'Re: 周末见个面?', 'to': 'jane@itcast.cn', 'body': '嘿 Jane! \n\n当然  
好啊, 好久不见! 你什么时候到? 具体哪天有空, 咱们约个时间喝咖啡! \n\n等你消息哈~\n\n虎  
哥'}, 'description': "Tool execution requires approval\n\nTool:  
send_email\nArgs: {'subject': 'Re: 周末见个面?', 'to': 'jane@itcast.cn',  
'body': '嘿 Jane! \n\n当然好啊, 好久不见! 你什么时候到? 具体哪天有空, 咱们约个时间喝咖  
啡! \n\n等你消息哈~\n\n虎哥'}"]}], 'review_configs': [{'action_name':  
'send_email', 'allowed_decisions': ['approve', 'edit', 'reject',  
'respond']}]}, id='fa58254d10e4de0106b56d0ed096063a'),), state=None,  
result=None),), interrupts=(Interrupt(value={'action_requests': [{'name':  
'send_email', 'args': {'subject': 'Re: 周末见个面?', 'to': 'jane@itcast.cn',  
'body': '嘿 Jane! \n\n当然好啊, 好久不见! 你什么时候到? 具体哪天有空, 咱们约个时间喝咖  
啡! \n\n等你消息哈~\n\n虎哥'}, 'description': "Tool execution requires  
approval\n\nTool: send_email\nArgs: {'subject': 'Re: 周末见个面?', 'to':  
'jane@itcast.cn', 'body': '嘿 Jane! \n\n当然好啊, 好久不见! 你什么时候到? 具体哪天  
有空, 咱们约个时间喝咖啡! \n\n等你消息哈~\n\n虎哥'}"]}], 'review_configs':
```

```
[{'action_name': 'send_email', 'allowed_decisions': ['approve', 'edit', 'reject', 'respond']}]}, id='fa58254d10e4de0106b56d0ed096063a'),))
```

2.8.3 历史消息处理

我们可以定义一个方法，专门从state中获取历史消息和Interrupt信息：

Code block

```
1  def get_messages(thread_id: str) -> dict:
2      """获取会话历史，如果存在中断则返回中断信息"""
3
4      config = {"configurable": {"thread_id": thread_id}}
5      state = agent.get_state(config)
6      if state is None or not state.values:
7          return {"messages": []}
8
9      messages = state.values.get("messages", [])
10
11     # 转换消息格式
12     result = []
13     for msg in messages:
14         if not msg.content:
15             continue
16         if isinstance(msg, HumanMessage):
17             result.append({"role": "user", "content": msg.content})
18         elif isinstance(msg, AIMessage):
19             result.append({"role": "assistant", "content": msg.content})
20
21     response = {"messages": result}
22
23     # 检查是否存在中断
24     interrupts = None
25     if hasattr(state, 'interrupts') and state.interrupts:
26         interrupts = state.interrupts
27     elif hasattr(state, 'tasks') and state.tasks:
28         for task in state.tasks:
29             if hasattr(task, 'interrupts') and task.interrupts:
30                 interrupts = task.interrupts
31                 break
32
33     if interrupts:
34         response["has_interrupt"] = True
35         response["interrupt"] = {
36             "reason": "需要人工确认",
```

```

37         "details": _serialize(interrupts)
38     }
39
40     return response
41
42 def _serialize(obj):
43     """递归转换对象为可 JSON 序列化的格式"""
44     if hasattr(obj, 'value'):
45         return _serialize(obj.value)
46     elif hasattr(obj, 'model_dump'):
47         return obj.model_dump()
48     elif isinstance(obj, (list, tuple)):
49         return [_serialize(item) for item in obj]
50     elif isinstance(obj, dict):
51         return {k: _serialize(v) for k, v in obj.items()}
52     return obj

```

测试：

Code block

```
1 get_messages("1")
```

结果：

Code block

```

1  {
2      "messages": [
3          {
4              "role": "user",
5              "content": "帮我查看下邮件。"
6          },
7          {
8              "role": "assistant",
9              "content": "我需要先验证您的身份才能帮您查看邮件。请提供您的
邮箱地址和密码以便进行身份验证。"
10         },
11         {
12             "role": "user",
13             "content": "huge@itcast.cn/123"
14         },
15         {
16             "role": "assistant",

```

```

17         "content": "好的，让我验证您的身份。"
18     },
19     {
20         "role": "assistant",
21         "content": "身份验证成功！现在让我查看您的收件箱。"
22     },
23     {
24         "role": "assistant",
25         "content": "好的，您的收件箱中有以下邮件：\n\n---\n\n###
E 邮件列表\n\n**1. 来自：** `jane@itcast.cn` (简) \n\n**主题：** 周末见个面？\n\n**状态：** ● 未读\n\n**内容：**\n> 嗨 虎哥，\n> 我下周会去城里，不知道我们有没有机会一起喝杯咖啡？\n>\n> 祝好，简\n\n---\n\n**2. 来自：** `lixiaolong@itcast.cn` (小李) \n\n**主题：** 周五会议\n\n**状态：** ✓ 已读\n\n**内容：**\n> 嗨 虎哥，\n> 非常抱歉，我周五的会议无法准时参加了，能不能重新安排个时间？\n>\n> 祝好，小李\n\n---\n\n您有一封来自 **简(jane@itcast.cn)** 的未读邮件，她想约您喝咖啡。您需要我回复哪封邮件吗？"
26     },
27     {
28         "role": "user",
29         "content": "帮我回复Jane，很高兴她能来，问问她具体见面时间"
30     },
31     {
32         "role": "assistant",
33         "content": "好的，我来回复Jane的邮件。"
34     },
35     {
36         "role": "assistant",
37         "content": "明白了，这是好朋友之间的邮件，我重新给您写一封更
自然的。"
38     }
39 ],
40 "has_interrupt": true,
41 "interrupt": {
42     "reason": "需要人工确认",
43     "details": [
44         {
45             "action_requests": [
46                 {
47                     "name": "send_email",
48                     "args": {
49                         "subject": "Re: 周末见个
面？",
50                         "body": "嗨 简，\n\n太好了！你要来城里啦，必须约起来喝杯咖啡！\n\n你具体啥时候方便呀？跟我说个时间呗~\n\n回头见！\n\n虎哥",
51                         "to": "jane@itcast.cn"
52                     }
53                 }

```



```
54         ],
55         "review_configs": [
56             {
57                 "action_name": "send_email",
58                 "allowed_decisions": [
59                     "approve",
60                     "edit",
61                     "reject",
62                     "respond"
63                 ]
64             }
65         ]
66     }
67 ]
68 }
69 }
```

3. 实战开发

OK，完成了基本的功能调试，接下来就是实战开发了。

与之前类似，我们还是基于FastAPI开发服务端接口，然后与提供好的前端对接。但有一点不同的是，我们之前都是调用LangChain的同步API，而本次实战中我们将采用异步API。

3.1 同步与异步

那么，同步与异步有什么区别呢？

- 同步API：Agent每次只能处理一个请求，如果有多个请求就会阻塞等待
- 异步API：Agent可以并行处理多个请求，并发能力更强

在LangChain中就同时提供了同步和异步两套API，以流式调用为例，同步与异步调用的API对比：

特性	agent.stream	agent.astream
编程范式	同步 (Synchronous)，使用 <code>for...in</code> 循环	异步 (Asynchronous)，需要在 <code>async for...in</code> 循环中调用

返回类型	一个标准的生成器 (Generator)	一个异步可迭代对象 (AsyncIterator)
核心用途	用于同步的、简单的脚本环境，如 Jupyter Notebook 或简单的命令行工具。	用于高性能的 Web 应用（如 FastAPI、Sanic），需要并发处理多个请求或事件循环的场景。

尽管方法名不同，但使用方式没有太大差别：

Code block

```
1  # 适用于同步环境
2  for chunk in agent.stream(
3      {"messages": [{"role": "user", "content": "讲解一下量子纠缠"}]},
4      stream_mode="messages",
5  ):
6      # 处理每一个 chunk
7      token, metadata = chunk
8      print(token.text, end="", flush=True)
```

Code block

```
1  # 适用于 FastAPI 等异步环境
2  async for chunk in agent.astream(
3      {"messages": [{"role": "user", "content": "讲解一下量子纠缠"}]},
4      stream_mode="messages",
5  ):
6      # 处理每一个 chunk
7      token, metadata = chunk
8      print(token.text, end="", flush=True)
```

3.2 异步Checkpointer

一旦使用了异步API，Agent的Checkpointer也必须使用异步版本。LangChain中同时提供了同步和异步版本的Checkpointer实现，完整列表参考官网文档：

 [https://docs.langchain.com/oss/python/langgraph/persistence#checkpoint...](https://docs.langchain.com/oss/python/langgraph/persistence#checkpoint-...)

Persistence - Docs by LangChain

Skip to main content Join us May 13th & May 14th at Interrupt, the Agent Conference by LangChain. Buy tickets > Docs by LangChain home page Open...

❗ See [checkpointer integrations](#) for available providers.

- `langgraph-checkpoint` : The base interface for checkpointer savers (`BaseCheckpointSaver`) and serialization/deserialization interface (`SerializerProtocol`). Includes in-memory checkpointer implementation (`InMemorySaver`) for experimentation. LangGraph comes with `langgraph-checkpoint` included.
- `langgraph-checkpoint-sqlite` : An implementation of LangGraph checkpointer that uses SQLite database (`SqliteSaver` / `AsyncSqliteSaver`). Ideal for experimentation and local workflows. Needs to be installed separately.
- `langgraph-checkpoint-postgres` : An advanced checkpointer that uses Postgres database (`PostgresSaver` / `AsyncPostgresSaver`), used in LangSmith. Ideal for using in production. Needs to be installed separately.
- `langchain-azure-cosmosdb` : An implementation of LangGraph checkpointer that uses Azure Cosmos DB for NoSQL (`CosmosDBSaverSync` / `CosmosDBSaver`). Ideal for using in production with Azure. Supports both sync and async operations, with Microsoft Entra ID authentication. Needs to be installed separately.

以SqliteSaver为例，LangChain提供了异步的AsyncSqliteSaver，官方的API文档如下：


Python / langgraph.checkpoint.sqlite
AsyncSqliteSaver

 <https://reference.langchain.com/python/langgraph.checkpoint.sqlite/aio/A...>
AsyncSqliteSaver | langgraph.checkpoint.sqlite
Python API reference for aio.AsyncSqliteSaver in langgraph.checkpoint.sqlite. Part of the LangChain ecosystem.

对于异步的数据库连接，虽然可以用 `async with` 来管理连接资源，但通常在生产中我们更多会使用连接池，所以就不建议用 `async with` 来处理了，而是建议手动管理连接。

示例代码：

Code block

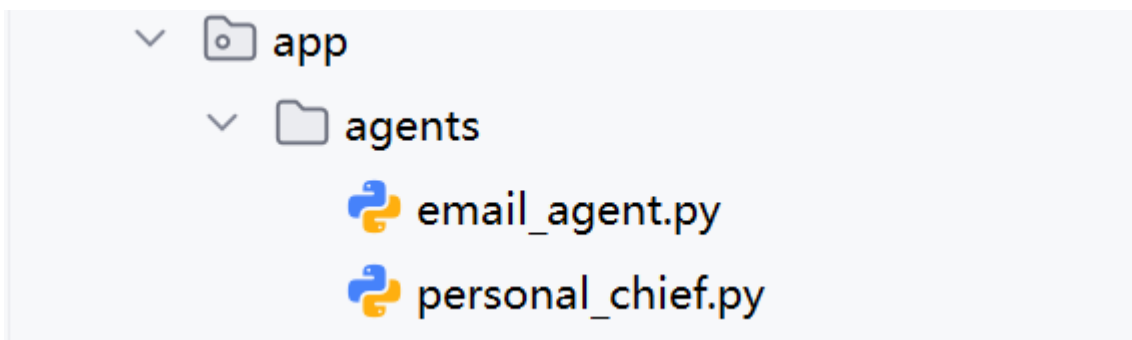
```

1  async def test():
2      # 创建连接
3      conn = await aioredis.connect("agent.db")
4      # 创建checkpointer
5      checkpointer = AsyncRedisSaver(conn=conn)
6
7      # 创建Agent
8      # 略..
9
10     # 关闭连接
11     await conn.close()

```

3.3 开发Agent

接下来，我们就可以正式开发Agent了，简单来说就是把之前jupyter中的代码拷贝过来。我们会把Agent代码封装到 `app.agents` 包下的 `email_agent` 文件：



不过，由于现在我们决定采用异步调用方式，并且还要支持HITL，隐藏其中有3个注意事项：

- 前端提交的信息不仅有message，可能还会包含用户对interrupt的决策
- 异步返回前端数据不仅有消息，还有Interrupt中断信息，不能仅返回data，而是要采用sse格式
- aioredis的初始化和使用必须在异步环境

3.3.1 重构ChatRequest

之前我们是纯对话的Agent，而这次包含了HITL，也就需要人工确认工具操作。因此前端不仅包含用户输入的message，还会出现interrupt的确认操作信息。

所以，我们必须重构ChatRequest类，添加新字段。

HITL人工确认的格式是这样的：

- approve

Code block

```
1 Command(  
2     resume={"decisions": [{"type": "approve"}]}  
3 )
```

- reject

Code block

```
1 Command(  
2     resume={  
3         "decisions": [  
4             {  
5                 "type": "reject",  
6                 # An explanation of why the request was rejected  
7                 "message": "算了，暂时不转了。"  
8             }  
9         ]  
10    }
```

- edit

Code block

```
1 Command(  
2     resume={  
3         "decisions": [  
4             {  
5                 "type": "edit",  
6                 # Edited action with tool name and args  
7                 "edited_action": {  
8                     # Tool name to call.  
9                     # Will usually be the same as the original action.  
10                    "name": "transfer_money",  
11                    # Arguments to pass to the tool.  
12                    "args": {"amount": 1000, "to": "王小明"},  
13                }  
14            }  
15        ]  
16    }
```

可以看到关键的 `decisions` 信息就是一个数组，其中的内容虽然不同，但都是Dict，在前端就是对象。所以我们可以让前端直接提交Dict格式即可。

另外，当用户提交的是interrupt_decision时，message就没有了，所以要变为可选参数。

因此，我们重构 `app.model.schemas.py` 中的 `ChatRequest`，内容如下：

Code block

```
1  from typing import Optional, Dict, Any, List
2
3  from pydantic import BaseModel
4
5  # --- 1. 数据模型 ---
6  class ChatRequest(BaseModel):
7      message: Optional[str] = None
8      image_url: Optional[str] = None
9      thread_id: str
10     # 用户确认的interrupt操作
11     interrupt_decision: Optional[Dict[str, Any]] = None
```

3.3.2 SSE

SSE（Server-Sent Events）是一种让服务器主动向浏览器推送数据的通信协议。

它基于 HTTP，使用 `text/event-stream` 格式，前端可以通过 `EventSource` API 方便地接收。其核心特点就是：服务器一次连接，多次推送，且前端可以自动重连，非常适合像大模型输出、Agent 逐步推理这类流式场景。

简单来说，就是**前端发起一次请求，服务端以stream方式不断推送新的数据给前端**，前端完成渲染，从而实现大模型的流式输出效果。

在标准的SSE中，服务端返回的数据格式是纯文本格式，可以包含event、data、id、retry字段，每个字段后跟冒号和空格，并以\n结束，不同消息之间则以两个\n结束

Code block

```
1  [field]: [value]\n
2  [field]: [value]\n
3  \n\n
```

支持的字段：

字段	说明	示例
event	指定事件类型（前端可监听特定事件）	event: message
data	实际数据内容（可以是多行，多行会拼接）	data: Hello
id	消息ID，客户端可用于断线重连	id: 1
retry	重连时间（毫秒），客户端断线后等待多久重新连接	retry: 3000

示例：

- 纯数据消息

Code block

```
1 data: 你好\n\n
```

- 带事件类型的消息

Code block

```
1 event: message \n\n2 data: {"content": "你好"}\n\n
```

在之前的Personal Chief案例中，我们采用了简化版本的SSE，直接返回数据：

```
# 流式调用Agent
for chunk, metadata in agent.stream(
    input={"messages": [message]},
    config={"configurable": {"thread_id": thread_id}},
    stream_mode="messages"
):
    if isinstance(chunk, AIMessageChunk) and chunk.content:
        yield chunk.content
```

但现在我们不仅有消息，还需要返回interrupt信息，所以必须指定类型，也就是event，然后是数据data，因此stream流处理代码是这样的：

```

1  async def generate_sse(self, thread_id: str, message: str, interrupt_decision:
2      dict):
3      """生成 SSE 事件流"""
4      config = {
5          "configurable": {
6              "thread_id": thread_id
7          }
8      }
9      # 使用 HumanMessage 封装用户消息
10     messages = {"messages": [HumanMessage(content=message)]}
11     # 判断是消息还是 command
12     _input = messages
13     if interrupt_decision:
14         _input = Command(resume={
15             "decisions": [interrupt_decision]
16         })
17
18     logger.info(f"调用agent, Input: {_input}")
19     try:
20         # 使用 astream_events 流式获取事件
21         async for chunk in self.agent.astream(
22             _input,
23             config=config,
24             stream_mode=["messages", "updates"],
25             version="v2"
26         ):
27             event_type = chunk["type"]
28             data = chunk["data"]
29
30             # 1. messages - LLM 流式输出 (token 级别)
31             if event_type == "messages":
32                 token, metadata = data
33                 content = None
34                 if isinstance(token, AIMessage) and hasattr(token, "content"):
35                     content = token.content
36
37                 if content:
38                     yield {
39                         "event": "message",
40                         "data": json.dumps(
41                             {"type": "message", "content": content},
42                             ensure_ascii=False
43                         )
44                     }
45
46             # 2. updates - 节点完成的 state 更新

```

```

47         elif event_type == "updates":
48             # 2a. 中断事件
49             if "__interrupt__" in data:
50                 interrupt_data = data['__interrupt__']
51                 details = _serialize(interrupt_data)
52                 yield {
53                     "event": "interrupt",
54                     "data": json.dumps(
55                         {
56                             "type": "interrupt",
57                             "interrupt": {
58                                 "reason": "需要人工确认",
59                                 "details": details
60                             }
61                         },
62                         ensure_ascii=False, default=str
63                     )
64                 }
65
66             # 流正常结束
67             yield {
68                 "event": "done",
69                 "data": json.dumps({"type": "done", "content": "处理完成"},
ensure_ascii=False)
70             }
71
72         except Exception as e:
73             logger.error(f"SSE 流中断: {e}", exc_info=True)
74             yield {
75                 "event": "error",
76                 "data": json.dumps({"type": "error", "error": str(e)},
ensure_ascii=False)
77             }

```

3.3.3 aiosqlite的初始化

首先，我们需要安装异步Sqlite依赖：

Code block

```
1 uv add aiosqlite
```


然后，需要注意的是，由于aiosqlite需要运行在异步环境中，因此我们不能像之前那样直接把代码写在模块中，而是要封装一个类，定义异步方法完成初始化。

由于我们是基于FastAPI开发，因此最好让Sqlite的生命周期与FastAPI生命周期一致。

为了方便在FastAPI的生命周期开始与结束时与Sqlite建立连接、销毁连接，我们需要将初始化aiosqlite、checkpointer、关闭Sqlite的代码封装为异步方法。

像这样：

Code block

```
1
2  class EmailAgent:
3
4      def __init__(self):
5          self.conn: aiosqlite.Connection = None
6          self.checkpointer: BaseCheckpointSaver = None
7          self.agent = None
8
9      async def init(self):
10         await self.init_checkpointer()
11         logger.info("checkpointer 初始化完成 ....")
12         await self.init_agent()
13         logger.info("email agent 初始化完成 ....")
14
15         async def init_checkpointer(self):
16             # 建立连接
17             os.makedirs("./db", exist_ok=True)
18             self.conn = await aiosqlite.connect("./db/mail_fiend.db")
19             logger.info("sqlite connection 完成 ....")
20             # 初始化checkpointer
21             self.checkpointer = AsyncSqliteSaver(conn = self.conn)
22             # 自动建表
23             await self.checkpointer.setup()
24
25         async def close(self):
26             # 关闭连接
27             await self.conn.close()
28             logger.info("sqlite connection 关闭 ....")
29
30         async def init_agent(self):
31             # 略
32             pass
33
```

3.3.4 完整Agent代码

所以Agent代码是这样的：

Code block

```
1  import json
2  from langchain.agents import AgentState, create_agent
3  from langchain.tools import tool, ToolRuntime
4  from langchain_core.messages import HumanMessage, AIMessage
5  from langgraph.checkpoint.base import BaseCheckpointSaver
6  from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver
7  from langgraph.types import Command
8  from langchain.messages import ToolMessage
9  from langchain.agents.middleware import wrap_model_call, dynamic_prompt,
   HumanInTheLoopMiddleware
10 from langchain.agents.middleware import ModelRequest, ModelResponse
11 from typing import Callable
12 from app.common.logger import logger
13 import aiosqlite
14 import os
15
16 AUTHENTICATED_KEY = "authenticated"
17
18 # state信息，记录用户是否授权邮箱操作
19 class AuthenticatedState(AgentState):
20     authenticated: bool
21
22 # ===== 1. 定义工具 =====
23 # 定义工具，用于用户邮箱鉴权
24 @tool
25 def authenticate(email: str, password: str, runtime: ToolRuntime) -> Command:
26     """Authenticate the user with the given email and password"""
27
28     # 定义变量，记录校验结果
29     authenticated = False
30     message = "Authentication failed"
31
32     # 校验邮箱和密码
33     if email == "huge@itcast.cn" and password == "123":
```

```

34         authenticated = True
35         message = "Successfully authenticated"
36
37         # 返回校验结果
38         return Command(
39             update={
40                 "authenticated": authenticated,
41                 "messages": [
42                     ToolMessage(message, tool_call_id=runtime.tool_call_id)
43                 ],
44             }
45         )
46     # 定义工具, 用于操作邮箱
47     @tool
48     def check_inbox() -> str:
49         """Read an email from the given address."""
50         # 模拟收件箱邮件
51         return [
52             {
53                 "subject": "周末见个面? ",
54                 "content": """
55                     嗨 虎哥,
56                     我下周会去城里, 不知道我们有没有机会一起喝杯咖啡?
57
58                     祝好, 简
59                 """,
60                 "from": "jane@itcast.cn",
61                 "status": "unread"
62             },
63             {
64                 "subject": "周五会议",
65                 "content": """
66                     嗨 虎哥,
67                     非常抱歉, 我周五的会议无法准时参加了, 能不能重新安排个时间?
68
69                     祝好, 小李
70                 """,
71                 "from": "lixiaolong@itcast.cn",
72                 "status": "checked"
73             }
74         ]
75
76     @tool
77     def send_email(to: str, subject: str, body: str) -> str:
78         """Send an response email"""
79         return f"邮件已发送至 {to} , 主题: {subject} , 内容: {body}"
80

```

```

81
82 # 定义中间件, 实现动态工具
83 @wrap_model_call
84 async def dynamic_tool_call(
85     request: ModelRequest, handler: Callable[[ModelRequest], ModelResponse]
86 ) -> ModelResponse:
87     """Allow read inbox and send email tools only if user provides correct
88     email and password"""
89
90     authenticated = request.state.get(AUTHENTICATED_KEY)
91
92     if authenticated:
93         tools = [check_inbox, send_email]
94     else:
95         tools = [authenticate]
96
97     request = request.override(tools=tools)
98     return await handler(request)
99
100 # 定义提示词
101 authenticated_prompt = "You are a helpful assistant that can check the inbox
102 and send emails."
103 unauthenticated_prompt = """You are a helpful email assistant.
104 For system security protocols, you must authenticate user before any other
105 interaction.
106 """
107
108 # 定义动态提示词中间件
109 @dynamic_prompt
110 def dynamic_prompt_func(request: ModelRequest) -> str:
111     """Generate system prompt based on authentication status"""
112     authenticated = request.state.get(AUTHENTICATED_KEY)
113     final_prompt = authenticated_prompt if authenticated else
114     unauthenticated_prompt
115     return final_prompt
116
117 # ===== 辅助函数 =====
118 def _serialize(obj):
119     """递归转换对象为可 JSON 序列化的格式"""
120     if hasattr(obj, 'value'):
121         return _serialize(obj.value)
122     elif hasattr(obj, 'model_dump'):
123         return obj.model_dump()
124     elif isinstance(obj, (list, tuple)):
125         return [_serialize(item) for item in obj]
126     elif isinstance(obj, dict):
127         return {k: _serialize(v) for k, v in obj.items()}

```

```
124         return obj
125
126
127     class EmailAgent:
128
129         def __init__(self):
130             self.conn: aiosqlite.Connection = None
131             self.checkpointer: BaseCheckpointSaver = None
132             self.agent = None
133
134         async def init(self):
135             await self.init_checkpointer()
136             logger.info("checkpointer 初始化完成 ....")
137             await self.init_agent()
138             logger.info("email agent 初始化完成 ....")
139
140         async def init_checkpointer(self):
141             # 建立连接
142             os.makedirs("./db", exist_ok=True)
143             self.conn = await aiosqlite.connect("./db/mail_fiend.db")
144             logger.info("sqlite connection 完成 ....")
145             # 初始化checkpointer
146             self.checkpointer = AsyncSqliteSaver(conn = self.conn)
147             # 自动建表
148             await self.checkpointer.setup()
149
150         async def close(self):
151             await self.conn.close()
152             logger.info("sqlite connection 关闭 ....")
153
154         async def init_agent(self):
155             self.agent = create_agent(
156                 "deepseek-chat",
157                 tools=[authenticate, check_inbox, send_email],
158                 state_schema=AuthenticatedState,
159                 checkpointer=self.checkpointer,
160                 middleware=[
161                     dynamic_tool_call,
162                     dynamic_prompt_func,
163                     HumanInTheLoopMiddleware(
164                         interrupt_on={
165                             "authenticate": False,
166                             "check_inbox": False,
167                             "send_email": True,
168                         }
169                     )
170                 ],
```

```

171         )
172
173
174     # ===== SSE 流式响应 =====
175     async def generate_sse(self, thread_id: str, message: str,
176 interrupt_decision: dict):
177         """生成 SSE 事件流"""
178         config = {
179             "configurable": {
180                 "thread_id": thread_id
181             }
182         }
183
184         # 使用 HumanMessage 封装用户消息
185         messages = {"messages": [HumanMessage(content=message)]}
186         # 判断是消息还是 command
187         _input = messages
188         if interrupt_decision:
189             _input = Command(resume={
190                 "decisions": [interrupt_decision]
191             })
192
193         logger.info(f"调用agent, Input: {_input}")
194         try:
195             # 使用 astream_events 流式获取事件
196             async for chunk in self.agent.astream(
197                 _input,
198                 config=config,
199                 stream_mode=["messages", "updates"],
200                 version="v2"
201             ):
202                 event_type = chunk["type"]
203                 data = chunk["data"]
204
205                 # 1. messages - LLM 流式输出 (token 级别)
206                 if event_type == "messages":
207                     token, metadata = data
208                     content = None
209                     if isinstance(token, AIMessage) and hasattr(token,
210 "content"):
211                         content = token.content
212
213                     if content:
214                         yield {
215                             "event": "message",
216                             "data": json.dumps(
217                                 {"type": "message", "content": content},

```

```

216         ensure_ascii=False
217     )
218 }
219
220     # 2. updates - 节点完成的 state 更新
221     elif event_type == "updates":
222         # 2a. 中断事件
223         if "__interrupt__" in data:
224             interrupt_data = data['__interrupt__']
225             details = _serialize(interrupt_data)
226             yield {
227                 "event": "interrupt",
228                 "data": json.dumps(
229                     {
230                         "type": "interrupt",
231                         "interrupt": {
232                             "reason": "需要人工确认",
233                             "details": details
234                         }
235                     },
236                     ensure_ascii=False, default=str
237                 )
238             }
239
240     # 无中断 → 流正常结束
241     yield {
242         "event": "done",
243         "data": json.dumps({"type": "done", "content": "处理完成"},
ensure_ascii=False)
244     }
245
246     except Exception as e:
247         logger.error(f"SSE 流中断: {e}", exc_info=True)
248         yield {
249             "event": "error",
250             "data": json.dumps({"type": "error", "error": str(e)},
ensure_ascii=False)
251         }
252
253     async def get_messages(self, thread_id: str) -> dict:
254         """获取会话历史, 如果存在中断则返回中断信息"""
255         logger.info(f"获取历史消息, thread_id: {thread_id}")
256
257         config = {"configurable": {"thread_id": thread_id}}
258         state = await self.agent.aget_state(config)
259         if state is None or not state.values:
260             return {"messages": []}

```

```

261
262     messages = state.values.get("messages", [])
263
264     # 转换消息格式
265     result = []
266     for msg in messages:
267         if not msg.content:
268             continue
269         if isinstance(msg, HumanMessage):
270             result.append({"role": "user", "content": msg.content})
271         elif isinstance(msg, AIMessage):
272             result.append({"role": "assistant", "content": msg.content})
273
274     response = {"messages": result}
275
276     # 检查是否存在中断
277     interrupts = None
278     if hasattr(state, 'interrupts') and state.interrupts:
279         interrupts = state.interrupts
280     elif hasattr(state, 'tasks') and state.tasks:
281         for task in state.tasks:
282             if hasattr(task, 'interrupts') and task.interrupts:
283                 interrupts = task.interrupts
284                 break
285
286     if interrupts:
287         response["has_interrupt"] = True
288         response["interrupt"] = {
289             "reason": "需要人工确认",
290             "details": _serialize(interrupts)
291         }
292
293     return response
294
295 email_agent = EmailAgent()
296
297 __all__ = ["email_agent"]

```

3.3.5 生命周期管理

现在，Agent中包含了两个异步的初始化和关闭方法：

- init：初始化Sqlite连接、checkpointer、agent
- close：关闭Sqlite连接

这两个方法应该分别在FastAPI启动时、关闭时调用。

所以，我们需要在 `main.py` 中通过lifespan来实现生命周期管理：

Code block

```
1  # 使用 lifespan 管理 checkpointer 生命周期
2  @asynccontextmanager
3  async def lifespan(app: FastAPI):
4      # 启动时初始化 checkpointer 和 agent
5      await email_agent.init()
6      yield
7      # 停机时，关闭连接
8      await email_agent.close()
9
10 app = FastAPI(
11     title="Heima Agent API",
12     description="黑马智能体API",
13     version="0.1.0",
14     lifespan=lifespan
15 )
```

3.4 对话接口

我们在 `app.v1.chat.py` 中添加一个新的接口，实现email_agent的对话功能：

Code block

```
1  from fastapi import APIRouter
2  from fastapi.responses import StreamingResponse
3  from sse_starlette.sse import EventSourceResponse
4  from app.agents.personal_chief import search_recipes, get_messages,
5  clear_messages
6  from app.models.schemas import ChatRequest
7  from app.agents.email_agent import email_agent
8
9
10 router = APIRouter()
11
12 @router.post("/chat/stream")
13 async def chat_endpoint(request: ChatRequest):
14     return StreamingResponse(
15         search_recipes(request.message, request.image_url, request.thread_id),
16         media_type="text/event-stream"
```

```

16     )
17
18
19 @router.get("/chat/messages")
20 async def get_chat_messages(thread_id: str):
21     """获取历史消息"""
22     messages = get_messages(thread_id)
23     return {"messages": messages}
24
25
26 @router.delete("/chat/messages")
27 async def clear_chat_messages(thread_id: str):
28     """清空历史消息"""
29     clear_messages(thread_id)
30     return {"success": True}
31
32
33 @router.post("/chat/send", tags=["聊天"])
34 async def send_chat(request: ChatRequest):
35     """发送聊天消息，使用 SSE 流式返回 Agent 响应。同时处理 HITL 中断决策。"""
36     return EventSourceResponse(email_agent.generate_sse(request.thread_id,
request.message or "", request.interrupt_decision))

```

3.5 会话管理接口

由于EmailFriend这个Agent的需求中包含会话管理的功能：



在LangChain中，会话是以 `thread_id` 来区分，所以会话管理功能都与`thread_id`有关：

- **新建会话**：需要给当前用户创建一个新会话，生成新的 `thread_id`
- **删除会话**：需要根据 `thread_id` 删除指定会话，并删除所有历史消息
- **查询会话列表**：查询所有会话，包含每个会话的`thread_id`、会话创建时间等信息

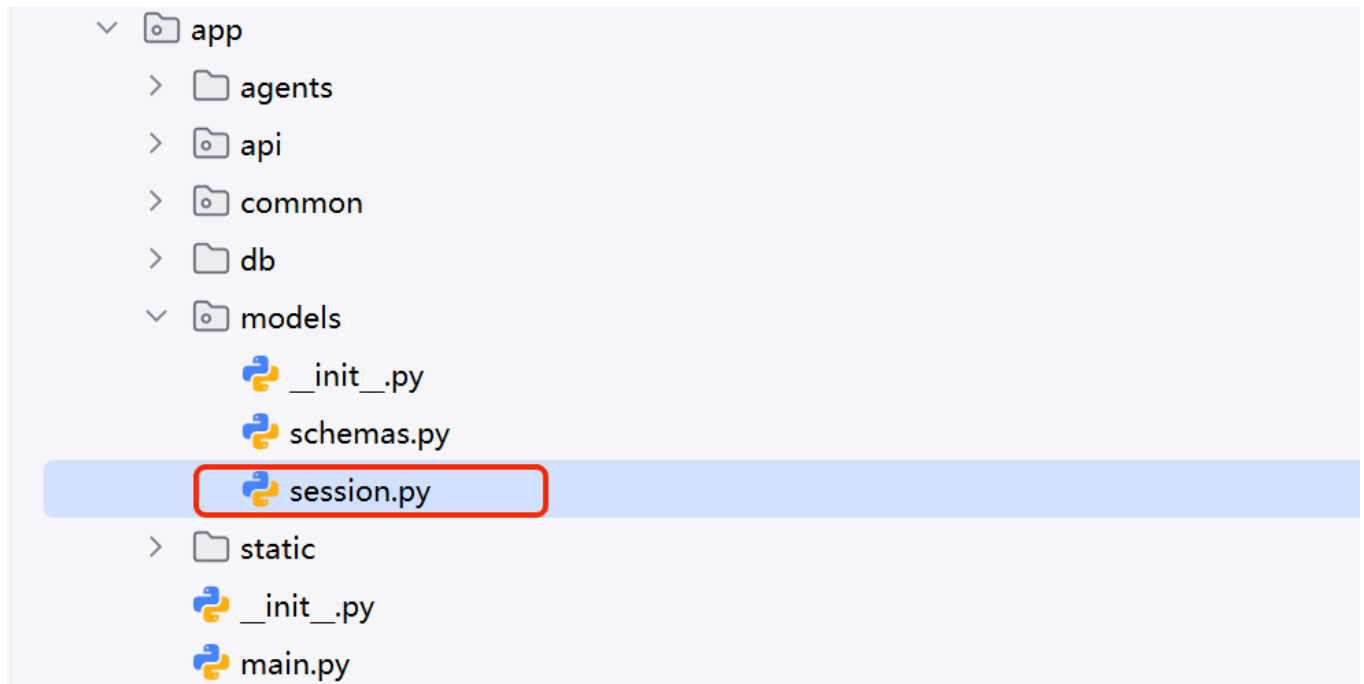
这些会话信息都需要保存到数据库中，我们同样是基于Sqlite来实现。

3.5.1 会话管理Model

首先，我们创建一个新的会话管理Model，在其中实现数据库的相关操作，包括：

- 创建表
- 新建会话
- 删除会话
- 查询会话列表

在 `app.models` 包含新建一个 `session.py` 文件：



内容如下：

Code block

```
1  import sqlite3
2  import uuid
3  from datetime import datetime
4  from typing import List, Optional
5  from pathlib import Path
6
7  from pydantic import BaseModel
8
9
10 # Pydantic 模型
11 class SessionCreate(BaseModel):
12     """创建会话的请求模型"""
13     user_id: str
14     biz_type: str
15     name: str
16
17
18 class SessionResponse(BaseModel):
19     """会话响应模型"""
20     thread_id: str
21     user_id: str
22     biz_type: str
23     name: str
24     created_at: str
25     updated_at: str
26
27
```

```
28 # 数据库配置
29 DB_PATH = Path(__file__).parent.parent / "db/sessions.db"
30
31
32 def get_db_connection():
33     """获取数据库连接"""
34     conn = sqlite3.connect(str(DB_PATH))
35     conn.row_factory = sqlite3.Row
36     return conn
37
38
39 def init_db():
40     """初始化数据库, 创建 sessions 表"""
41     conn = get_db_connection()
42     cursor = conn.cursor()
43     cursor.execute("""
44         CREATE TABLE IF NOT EXISTS sessions (
45             thread_id TEXT PRIMARY KEY,
46             user_id TEXT NOT NULL,
47             biz_type TEXT NOT NULL,
48             name TEXT NOT NULL,
49             created_at TEXT NOT NULL,
50             updated_at TEXT NOT NULL
51         )
52     """)
53     conn.commit()
54     conn.close()
55
56
57 def create_session(session: SessionCreate) -> SessionResponse:
58     """创建新会话"""
59     conn = get_db_connection()
60     cursor = conn.cursor()
61
62     thread_id = str(uuid.uuid4())
63     now = datetime.now().isoformat()
64
65     cursor.execute(
66         """
67         INSERT INTO sessions (thread_id, user_id, biz_type, name, created_at,
68         updated_at)
69         VALUES (?, ?, ?, ?, ?, ?)
70         """,
71         (thread_id, session.user_id, session.biz_type, session.name, now, now)
72     )
73     conn.commit()
74     conn.close()
```

```
74
75     return SessionResponse(
76         thread_id=thread_id,
77         user_id=session.user_id,
78         biz_type=session.biz_type,
79         name=session.name,
80         created_at=now,
81         updated_at=now
82     )
83
84
85 def get_sessions(user_id: Optional[str] = None, biz_type: Optional[str] =
None) -> List[SessionResponse]:
86     """查询会话列表, 支持按 user_id 和 biz_type 筛选"""
87     conn = get_db_connection()
88     cursor = conn.cursor()
89
90     query = "SELECT thread_id, user_id, biz_type, name, created_at, updated_at
FROM sessions"
91     params = []
92
93     if user_id or biz_type:
94         conditions = []
95         if user_id:
96             conditions.append("user_id = ?")
97             params.append(user_id)
98         if biz_type:
99             conditions.append("biz_type = ?")
100             params.append(biz_type)
101         query += " WHERE " + " AND ".join(conditions)
102
103     query += " ORDER BY updated_at DESC"
104
105     cursor.execute(query, params)
106     rows = cursor.fetchall()
107     conn.close()
108
109     return [
110         SessionResponse(
111             thread_id=row["thread_id"],
112             user_id=row["user_id"],
113             biz_type=row["biz_type"],
114             name=row["name"],
115             created_at=row["created_at"],
116             updated_at=row["updated_at"]
117         )
118         for row in rows
```

```

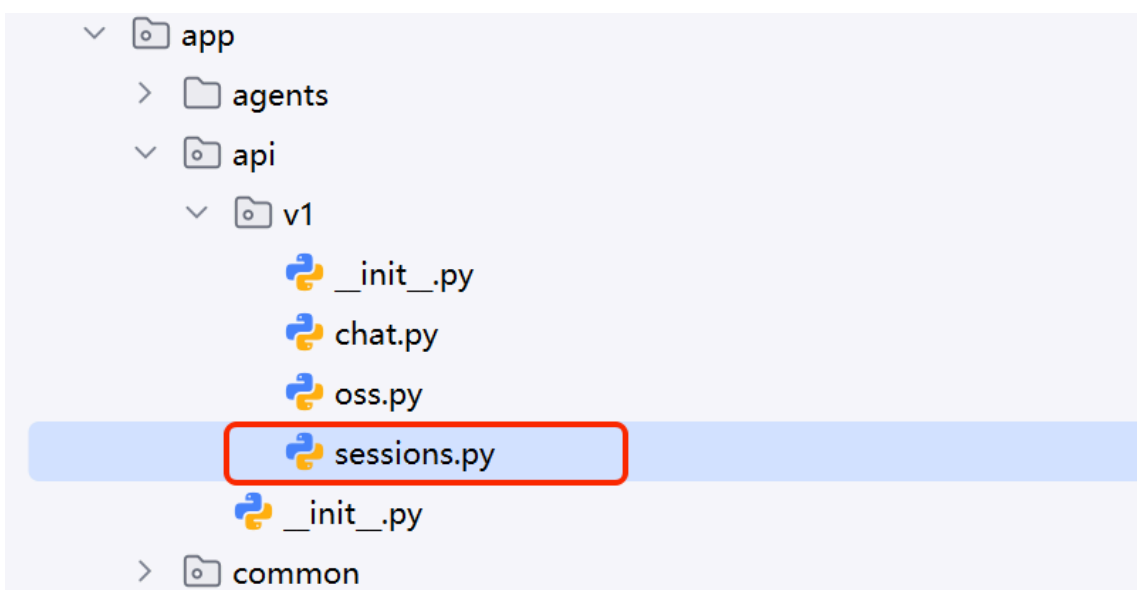
119     ]
120
121
122     def delete_session(thread_id: str) -> bool:
123         """删除会话"""
124         conn = get_db_connection()
125         cursor = conn.cursor()
126
127         cursor.execute("DELETE FROM sessions WHERE thread_id = ?", (thread_id,))
128         deleted = cursor.rowcount > 0
129         conn.commit()
130         conn.close()
131
132         return deleted
133
134
135     # 初始化数据库
136     init_db()

```

3.5.2 会话管理接口

接下来，我们还要开发基于FastAPI的接口，供前端使用。

在 `app.api.v1` 包下新建一个 `sessions.py` 文件，



内容如下：

Code block

```
1  from typing import List, Optional
2
3  from fastapi import APIRouter, HTTPException, Query
4
5  from app.models.session import (
6      SessionCreate,
7      SessionResponse,
8      get_sessions,
9      create_session,
10     delete_session
11 )
12 from app.agents.email_agent import email_agent
13
14 router = APIRouter()
15
16
17 @router.post("/sessions", response_model=SessionResponse, tags=["会话"])
18 def create_new_session(session: SessionCreate):
19     """创建新会话"""
20     return create_session(session)
21
22
23 @router.get("/sessions", response_model=List[SessionResponse], tags=["会话"])
24 def list_sessions(
25     user_id: Optional[str] = Query(None, description="用户ID"),
26     biz_type: Optional[str] = Query(None, description="业务类型")
27 ):
28     """查询会话列表, 支持按 user_id 和 biz_type 筛选"""
29     return get_sessions(user_id=user_id, biz_type=biz_type)
30
31
32 @router.delete("/sessions/{thread_id}", tags=["会话"])
33 async def remove_session(thread_id: str):
34     """删除会话及其checkpoint"""
35     deleted = delete_session(thread_id)
36     if not deleted:
37         raise HTTPException(status_code=404, detail="Session not found")
38     # 同步删除 LangGraph checkpoint 中的会话状态
39     await email_agent.clear_messages(thread_id)
40     return {"message": "Session deleted successfully"}
41
42
43 @router.get("/sessions/{thread_id}/messages", tags=["会话"])
44 async def get_session_messages(thread_id: str):
45     """获取会话的历史消息"""
46     try:
47         result = await email_agent.get_messages(thread_id)
```



```
48         return result
49     except Exception as e:
50         return {"messages": [], "error": str(e)}
```

3.5.3 挂载router

最后，在 `main.py` 中挂载我们刚刚写的 `sessions.py`：

Code block

```
1  from app.api.v1 import sessions
2
3  # 略
4
5  app.include_router(sessions.router, prefix="/api/v1", tags=["会话管理"])
```

`main.py`的完整代码：

Code block

```
1  import os
2  from fastapi import FastAPI
3  from fastapi.responses import FileResponse
4  from fastapi.staticfiles import StaticFiles
5  from fastapi.middleware.cors import CORSMiddleware
6  from app.api.v1 import chat
7  from app.api.v1 import oss
8  from app.api.v1 import sessions
9  from app.common.logger import setup_logging
10 from contextlib import asynccontextmanager
11 from app.agents.email_agent import email_agent
12 # 初始化日志配置
13 setup_logging()
14
15 from dotenv import load_dotenv
16 load_dotenv()
17
18 # 使用 lifespan 管理 checkpointer 生命周期
19 @asynccontextmanager
20 async def lifespan(_app: FastAPI):
21     # 启动时初始化 checkpointer 和 agent
22     await email_agent.init()
```

```
23     yield
24     # 停机时, 关闭连接
25     await email_agent.close()
26
27 app = FastAPI(
28     title="Personal Chief API",
29     description="私厨",
30     version="0.1.0",
31     lifespan=lifespan
32 )
33
34 # 1. 配置跨域资源共享 (CORS)
35 # 插件开发中, 由于请求来自浏览器扩展环境, 必须正确配置 CORS
36 app.add_middleware(
37     CORSMiddleware,
38     allow_origins=["*"], # 生产环境建议指定插件的 ID 或具体域名
39     allow_credentials=True,
40     allow_methods=["*"],
41     allow_headers=["*"],
42 )
43
44 # 2. 挂载路由
45 app.include_router(chat.router, prefix="/api/v1", tags=["对话"])
46 app.include_router(oss.router, prefix="/api/v1", tags=["申请上传签名url"])
47 app.include_router(sessions.router, prefix="/api/v1", tags=["会话管理"])
48
49 # 3. 挂载前端资源
50 static_dir = os.path.join(os.path.dirname(__file__), "static")
51 if os.path.exists(static_dir):
52     app.mount("/", StaticFiles(directory=static_dir, html=True), name="static")
53
54 # 前端 fallback 路由 - 只处理非 API 请求
55 @app.get("/{path:path}", include_in_schema=False)
56 async def serve_frontend(path: str):
57     # 排除 API 路径
58     if path.startswith("api/"):
59         from fastapi.responses import JSONResponse
60         return JSONResponse({"error": "Not Found"}, status_code=404)
61     # 如果请求的是静态文件, 直接返回
62     file_path = os.path.join(static_dir, path)
63     if os.path.isfile(file_path):
64         return FileResponse(file_path)
65     # 否则返回 index.html (SPA fallback)
66     index_path = os.path.join(static_dir, "index.html")
67     if os.path.exists(index_path):
68         return FileResponse(index_path)
69     return {"message": "你的独家私厨上线了~", "status": "ok"}
```

```
70
71 if __name__ == "__main__":
72     import uvicorn
73     # 启动命令: python -m app.main
74     uvicorn.run("app.main:app", host="127.0.0.1", port=8001, reload=True)
```

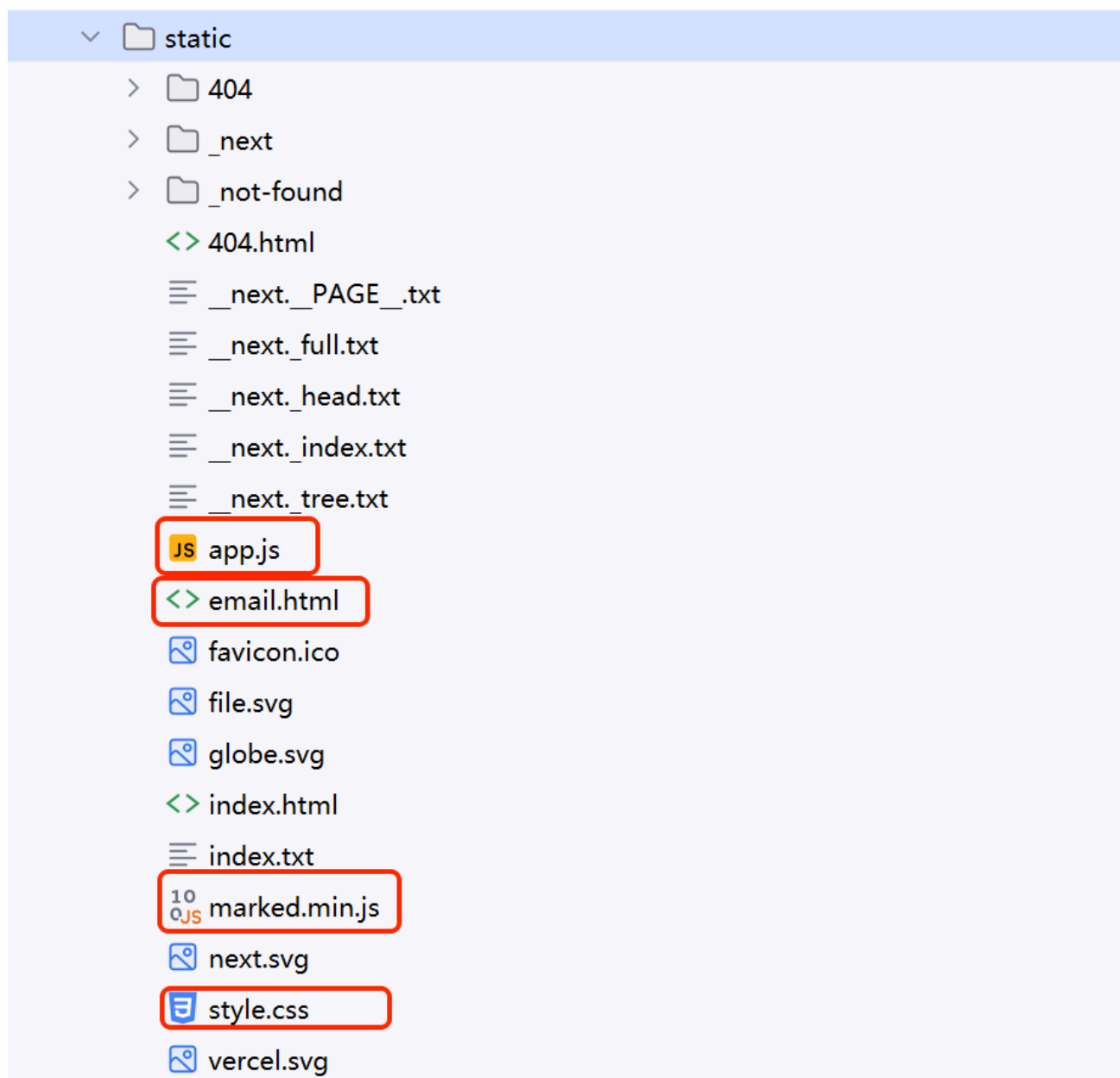
3.6 前端

在课前资料中提供好了MailFriend的前端代码：

> 此电脑 > Data (D:) > 课程资料 > langchain开发指南 > 资料 > mail-friend

名称	类型	大小
 app.js	JavaScript 文件	18 KB
 email.html	Microsoft Edge ...	6 KB
 marked.min.js	JavaScript 文件	39 KB
 style.css	CSS 文件	14 KB

复制代码，直接粘贴到项目的 `app/static` 目录即可：



3.7 测试

启动服务，访问：<http://127.0.0.1:8081/email.html>，即可看到MailFriend的页面：

 MailFriend

+ 新建会话

• 今天 23:25

新会话 3

今天 12:04

新会话 2

今天 11:32

新会话 1

● 新会话 3

输入消息...



